

DOCUMENT INVESTIGATOR

Complete Build Specification

ALGOTHON'26 | Problem Statement ID: ALG-AI-02 | Domain: AI / ML

What this document is. A single, self-contained specification that a development team, AI agency, or AI coding agent (for example Anti-Gravity, Cursor, or similar) can follow to build the full product from start to finish. It defines what to build, how to build it, which technologies to use, the exact approach for every component, the standout features, the demo data, the tests, and the final deliverables. Where a choice is left open, a default is stated so no guessing is needed.

Instruction to the builder. Build exactly to this specification. Implement all items marked **MUST** first, then the items marked **STANDOUT** in the order listed in Section 14. Do not replace the core idea with a simple 'chat with PDF' app. The three things that win this problem are: grounded citations, visible conflict detection, and honest uncertainty. Every other feature supports those three.

Contents

1. Product Overview	3
2. Requirements Traceability	3
3. System Architecture	4
4. Technology Stack	4
5. Project Structure	5
6. Data Model	5
7. Component Specifications	6
8. Standout Features Catalog	9
9. API Specification	10
10. Frontend and UX Specification	10
11. Demo Dataset Specification	11
12. Testing and Evaluation	11
13. Configuration, Security, and Privacy	12
14. Build Order and Definition of Done	13
15. Submission Deliverables	13

1. Product Overview

1.1 The problem

Information is scattered across PDFs, scanned images, and text files. People need answers without reading every document, and they need to know when documents disagree or when the answer simply is not in the documents. Plain AI chat tools fail here because they answer confidently even when sources conflict or are missing.

1.2 The product

Document Investigator is a web platform where a user uploads multiple documents, asks natural-language questions, and receives answers that: (a) cite the exact document, page, and passage; (b) show conflicts between documents side by side instead of hiding them; (c) state clearly when the evidence is weak or missing; and (d) can be saved into an exportable investigation report.

1.3 Target users

- Investigators, auditors, legal and compliance staff, procurement teams, journalists, and students who work with many documents.
- Hackathon judges, who need to understand the value within the first two minutes of a demo.

1.4 Success criteria (how the project is judged)

Judging criterion	Weight	How this product earns it
Functionality and completion	30%	Upload, extract, index, ask, cite, detect conflict, show uncertainty all work end to end.
Technical implementation	20%	Hybrid retrieval, reranking, structured JSON outputs, claim verification, clean architecture.
Innovation and problem understanding	20%	Conflict Panel, deterministic fact cross-checks, abstention, document trust scoring.
UX and presentation	15%	Split-screen source viewer with highlights, confidence badges, investigation board.
Testing, edge cases, reliability	15%	Golden question set, edge-case suite, evaluation metrics shown in the app.

1.5 Non-goals

- No user accounts or payments are required. No training of custom models. No support for video or audio files.

2. Requirements Traceability

Each mandatory item from the problem statement is mapped to a feature and the section that specifies it.

Problem statement 'Must Have'	Feature that satisfies it	Section
Multiple document formats	PDF, scanned image, TXT, DOCX, MD upload with auto-detection	7.1
Extraction / indexing	Text extraction, OCR fallback, chunking, hybrid vector plus keyword index	7.1 - 7.3
Natural-language Q&A	Retrieval, reranking, grounded LLM answer	7.4 - 7.5
Source / section references	Clickable citations with page, section, highlighted passage	7.6
Conflict detection	LLM contradiction analysis plus deterministic number/date cross-check	7.7

Problem statement 'Must Have'	Feature that satisfies it	Section
Uncertainty handling	Confidence score, abstention, 'what is missing' explanation	7.8
Bonus: conflicting docs and uncertainty	Conflict Panel plus confidence badge on every answer	7.7, 7.8

3. System Architecture

The system has five layers. The browser talks only to the FastAPI backend. The backend orchestrates parsing, retrieval, and LLM calls. All state lives in SQLite (metadata, answers, notes) and ChromaDB (vectors).

FRONTEND (Next.js + Tailwind)	Upload page Chat and Conflict Panel PDF source viewer Timeline and Entity view Evidence Board Report export
API LAYER (FastAPI, REST + SSE streaming)	/documents /ask /conflicts /entities /timeline /notes /report /eval
INTELLIGENCE LAYER (Python services)	Query rewriter Hybrid retriever Reranker Answer generator Conflict detector Fact cross-checker Claim verifier Confidence scorer
INGESTION LAYER (Python services)	File type detection PDF text extraction OCR Layout and page mapping Chunker Entity and date extractor Reliability scorer Dedup and hashing
STORAGE	SQLite (documents, chunks, entities, answers, notes, audit log) ChromaDB (embeddings) BM25 index (in memory, rebuilt on upload) Local file store (originals)

3.1 End-to-end data flow for one question

1. User asks a question in the chat box.
2. Query rewriter expands the question (synonyms, resolves follow-up references).
3. Hybrid retriever gets top 30 chunks (vector search + BM25), merges with RRF.
4. Reranker scores the 30 chunks and keeps the best 8.
5. Conflict detector compares the 8 chunks and returns structured JSON.
6. Fact cross-checker compares extracted numbers/dates across documents (rule based).
7. Answer generator writes the answer using ONLY the 8 chunks, with inline citation IDs.
8. Claim verifier splits the answer into claims and checks each against its cited chunk.
9. Confidence scorer combines signals into High / Medium / Low, or triggers abstention.
10. API streams back: answer, citations, conflicts, confidence, follow-up questions.
11. Everything is written to the audit log (question, chunks, prompts, model, output).

4. Technology Stack

Defaults are chosen for speed of development and reliability in a live demo. Equivalent technologies are allowed by the competition rules.

Layer	Default choice	Why	Acceptable alternative
Backend framework	Python 3.11 + FastAPI	Fast to build, async, auto API docs	Flask
Frontend	Next.js 14 + React + Tailwind CSS	Rich UI, split-screen viewer	Streamlit (fastest fallback)
PDF text extraction	PyMuPDF (fitz)	Fast, gives page and bounding boxes	pdfplumber
OCR	Tesseract (eng+hin) via pytesseract; EasyOCR fallback	Free, supports Hindi and English	PaddleOCR
DOCX / TXT / MD	python-docx, plain readers	Simple and reliable	docx2txt

Layer	Default choice	Why	Acceptable alternative
Embeddings	BAAI/bge-small-en-v1.5 (or multilingual-e5-small) via sentence-transformers	Free, local, good quality	OpenAI or Gemini embeddings
Vector store	ChromaDB (persistent)	Zero setup	FAISS
Keyword search	rank_bm25	Catches exact numbers and names	SQLite FTS5
Reranker	BAAI/bge-reranker-base (cross-encoder)	Big quality gain for little work	LLM-based rerank
LLM	Claude, GPT, or Gemini via API (one config switch)	Strong JSON and reasoning output	Local Llama via Ollama
Entity extraction	spaCy + LLM JSON extraction + regex for money and dates	Combines recall and precision	LLM only
Database	SQLite + SQLAlchemy	No server needed	PostgreSQL
PDF viewer	react-pdf (PDF.js) with highlight overlay	Page jump and text highlight	iframe + #page=
Report export	reportlab or WeasyPrint (PDF), plain Markdown	Simple and offline	docx library
Testing	pytest, httpx, Playwright (one smoke test)	Covers API and UI	unittest
Deployment	Docker Compose; frontend on Vercel, backend on Render or Railway	One-command run plus live demo	Local only

5. Project Structure

```

document-investigator/
  README.md           # setup, architecture diagram, limitations, AI disclosure
  docker-compose.yml
  .env.example
  backend/
    app/
      main.py          # FastAPI app and routers
      config.py        # env vars, thresholds
      db.py models.py  # SQLAlchemy models
      routers/ documents.py ask.py conflicts.py entities.py notes.py report.py eval.py
      ingestion/ detect.py pdf.py ocr.py docx.py chunker.py reliability.py dedup.py
      intelligence/ rewrite.py retrieve.py rerank.py answer.py conflict.py
                  crosscheck.py verify.py confidence.py followups.py
      extraction/ entities.py dates.py money.py timeline.py
      llm/ client.py prompts.py schemas.py      # all prompts live in prompts.py
      report/ builder.py
      tests/ test_ingestion.py test_retrieval.py test_conflict.py test_edge_cases.py
      eval/ golden_set.json run_eval.py
  frontend/
    app/ page.tsx (upload) workspace/page.tsx report/page.tsx
    components/ ChatPanel.tsx ConflictPanel.tsx SourceViewer.tsx ConfidenceBadge.tsx
                  EntityList.tsx Timeline.tsx EvidenceBoard.tsx DocTrustCard.tsx
  demo_data/          # the planted-conflict vendor dispute documents
  docs/ architecture.png decisions.md test_evidence.md

```

6. Data Model

Table	Key fields	Purpose
documents	id, filename, file_type, sha256, uploaded_at, page_count, doc_date, doc_type, authority_level, ocr_quality, reliability_score, status	One row per uploaded file

Table	Key fields	Purpose
chunks	id, document_id, page, section_title, text, char_start, char_end, bbox_json, token_count, is_ocr, chroma_id	Searchable passages with exact location
entities	id, document_id, chunk_id, type (PERSON, ORG, DATE, AMOUNT, PERCENT, PLACE), value, normalized_value	People, organizations, money, dates
facts	id, document_id, chunk_id, subject, predicate, value, unit, normalized_value, as_of_date	Structured claims such as 'delivery_date = 2026-02-15' used for cross-checking
conversations / messages	id, question, answer, confidence, confidence_band, created_at, abstained	Q&A history
citations	id, message_id, chunk_id, quote, supports_claim	Which passages support which part of an answer
conflicts	id, message_id, topic, side_a_chunk, side_b_chunk, side_a_value, side_b_value, severity, explanation	Detected disagreements
notes	id, message_id, text, pinned, created_at	Evidence Board items
audit_log	id, ts, event, payload_json, model, prompt_version	Reproducibility and trust

7. Component Specifications

7.1 Ingestion and extraction (MUST)

- **Accepted types:** .pdf, .png, .jpg, .jpeg, .tiff, .txt, .md, .docx. Maximum 25 MB per file. Detect type by content (magic bytes), not only extension.
- **Native PDF:** extract text per page with PyMuPDF, keeping text blocks with bounding boxes so citations can be highlighted.
- **Scanned PDF or image:** if a page yields fewer than 30 characters, render it at 300 DPI and run OCR (Tesseract, languages eng+hin). Store per-page OCR confidence.
- **DOCX:** read paragraphs and tables; treat each heading as a section boundary. **TXT/MD:** split by blank lines and headings.
- **Metadata:** detect document date (from text or file metadata), document type (contract, invoice, email, meeting notes, receipt, other) with a short LLM classification call.
- **Failure handling:** corrupted, password-protected, or empty files must produce a clear status message (status = failed, reason shown in UI) and must never crash the batch.

7.2 Chunking (MUST)

- Chunk size 350 tokens with 60-token overlap, but never split in the middle of a table row or a numbered clause. Prefer splitting at headings and paragraph ends.
- Each chunk stores document, page, section title, character offsets, and bounding box. A chunk that spans two pages stores both pages.
- Prefix every chunk's embedding text with its context line: 'Document: Contract.pdf | Section: Payment Terms | Page 3'. This improves retrieval noticeably.

7.3 Indexing (MUST)

- Embed every chunk and upsert into ChromaDB with metadata (doc_id, page, section, doc_type, doc_date).
- Maintain a BM25 index over the same chunks so exact tokens such as '4,50,000' or 'INV-2041' are found even if embeddings miss them.
- Re-indexing a document must be idempotent (same sha256 means skip and tell the user it is a duplicate).

7.4 Retrieval and reranking (MUST)

- **Query rewrite:** one fast LLM call turns follow-ups into standalone questions and adds 2 alternative phrasings.
- **Hybrid search:** top 20 from vectors plus top 20 from BM25, merged with Reciprocal Rank Fusion (k = 60).
- **Rerank:** cross-encoder scores the merged set; keep the top 8. Store the top rerank score because it feeds the confidence calculation.
- **Coverage rule:** if the question mentions several documents or compares things, force at least one chunk from each mentioned document into the final set.

7.5 Grounded answer generation (MUST)

The LLM receives only the selected chunks, each labelled with an ID. It must answer from those chunks alone and cite IDs inline. Use this system prompt as the default:

You are a document investigator. Answer ONLY using the numbered passages provided.

Rules:

1. Every factual sentence must end with citation IDs like [C3] or [C3][C5].
2. If the passages do not contain the answer, reply exactly: INSUFFICIENT_EVIDENCE and list what information is missing.
3. If passages disagree, do NOT pick a side silently. State both values with their sources and say they conflict.
4. Never use outside knowledge. Never invent numbers, names, or dates.
5. Be concise. Use short paragraphs or bullets.

Return JSON: {"answer": str, "used_chunks": [ids], "missing_info": [str]}

7.6 Citations and source viewer (MUST)

- Every citation in the answer is a clickable chip showing 'Document name, p. N'. Clicking opens the original file in the right-hand viewer at that page and highlights the exact passage using the stored bounding box (for OCR pages, highlight the OCR line boxes).
- Hovering a chip shows the quoted passage in a tooltip. Images are shown with a rectangle overlay.
- If a passage cannot be located (rare), fall back to showing the page and the text in a side card. Never show a citation that points to nothing.

7.7 Conflict detection (MUST, core differentiator)

Conflict detection runs on two independent paths. A conflict is reported if either path finds one.

Path A - LLM contradiction analysis

After reranking, send the 8 chunks to the LLM with this task and require strict JSON:

Task: find statements in the passages that make claims about the SAME topic.

For each topic return whether the claims AGREE, CONTRADICT, or are UNRELATED.

JSON schema:

```
{"topics":[{"topic":"delivery date","status":"CONTRADICT",
  "claims":[{"chunk":"C2","doc":"Contract.pdf","value":"15 Feb 2026","quote":"..."},
    {"chunk":"C5","doc":"Email_Vendor.docx","value":"20 Feb 2026","quote":"..."}],
  "severity":"HIGH|MEDIUM|LOW","explanation":"one sentence"}]}
```

Only report CONTRADICT when the claims cannot both be true. Different dates for different events are NOT contradictions.

Path B - Deterministic fact cross-check

- At ingestion, extract structured facts (amounts, dates, percentages, quantities, party names) with regex plus LLM, normalise them (for example 'Rs 4,50,000' becomes 450000 INR; '20th Feb 2026' becomes 2026-02-20) and store them in the facts table.
- At question time, group facts by subject and predicate. If two documents give different normalised values for the same subject and predicate, create a conflict record with both sources.
- This path catches numeric conflicts the LLM may overlook and gives you a reliable, explainable result for the demo.

Conflict resolution policy

- Never hide a conflict. Show the answer with a banner: 'Documents disagree'.
- Suggest a likely authoritative source using document trust score (Section 7.11): signed contract above email above meeting notes; newer above older when the same document type.
- Present 'most likely' as a suggestion with reasoning, never as a fact.

7.8 Confidence and uncertainty (MUST)

Compute a confidence score from 0 to 1 for every answer:

```
confidence = 0.40 * retrieval_strength      # top rerank score, normalised 0..1
            + 0.25 * claim_support          # share of answer claims verified (7.9)
            + 0.20 * source_agreement       # 1.0 if all cited chunks agree, lower otherwise
            + 0.15 * doc_reliability        # mean reliability of cited documents (7.11)
if conflict_found: confidence -= 0.25
bands: >= 0.75 HIGH | 0.50 - 0.74 MEDIUM | < 0.50 LOW
```

- **Abstention:** if the top rerank score is below the configured threshold (default 0.25) or the LLM returns INSUFFICIENT_EVIDENCE, the app replies 'The uploaded documents do not contain this information', lists the missing information, and suggests which kind of document would answer it.
- **Badge:** every answer shows a coloured badge (green, amber, red) with a 'Why this confidence?' popover listing the four signals and the penalty.
- **Calibration:** thresholds are set in config.py and tuned on the golden set (Section 12).

7.9 Claim-level verification (STANDOUT)

- Split the final answer into atomic claims (one LLM call or sentence splitter).
- For each claim, check it against its cited chunk using an entailment prompt: SUPPORTED, PARTIAL, or UNSUPPORTED.
- Unsupported claims are removed or shown with a warning marker. The UI shows 'Verified claims: 5 of 6'.
- This is the anti-hallucination story to show judges.

7.10 Entities and timeline (STANDOUT)

- Extract PERSON, ORG, DATE, AMOUNT, PLACE with spaCy plus LLM JSON extraction; merge duplicates (for example 'Northwind' and 'Northwind Supplies Pvt Ltd').
- **Entity view:** a searchable list; clicking an entity lists every document and passage that mentions it.
- **Timeline view:** all dated events from all documents on one horizontal timeline, colour-coded by document. Dated events that disagree are drawn with a red marker.

7.11 Document reliability and trust score (STANDOUT)

```
reliability = 0.35 * authority      # contract/signed=1.0, invoice=0.8, email=0.6, notes=0.4
            + 0.25 * ocr_quality    # 1.0 for native text, else mean OCR confidence
            + 0.20 * completeness  # 0 if pages missing or truncated text detected
            + 0.20 * recency        # newer document of same type scores higher
```

- Show a Trust Card per document with the score and the reasons (for example 'Blurry scan, OCR 62%'). Low-trust sources lower answer confidence.

7.12 Investigation report export (STANDOUT)

- One-click export to PDF and Markdown containing: case title, document list with trust scores, every saved question and answer, citations with page numbers, all conflicts, unresolved questions, and the audit trail summary.
- Include a first page 'Key findings' that lists the top conflicts and open gaps.

8. Standout Features Catalog

These features separate the project from the many 'chat with your PDF' submissions. Priority: P1 build first, P2 build next, P3 only if everything else is stable.

#	Feature	What the user sees	How it works	Priority
1	Conflict Panel	Side-by-side 'Document A says X, Document B says Y' with severity and highlighted sources	Section 7.7, two detection paths	P1
2	Confidence badge with explanation	Green/amber/red badge and 'Why?' breakdown on every answer	Section 7.8	P1
3	Abstention with missing-info list	'Not found in documents. Missing: warranty terms' instead of a guess	INSUFFICIENT_EVIDENCE path plus threshold	P1
4	Claim-level verification	'Verified claims 5 of 6', unsupported claim flagged	Section 7.9	P1
5	Exact-passage source highlighting	Click a citation, the PDF opens on the page with the text highlighted	Stored bounding boxes, react-pdf overlay	P1
6	Hybrid search plus reranker	Noticeably more accurate answers, including exact numbers	Section 7.4	P1
7	Deterministic numeric and date cross-check	Mismatch alerts such as advance paid differs from contract terms	Facts table comparison	P1
8	Entity and timeline view	Who/what/when across all files on one timeline	Section 7.10	P2
9	Document Trust Cards	Reliability score per file with reasons	Section 7.11	P2
10	Evidence Board	Pin answers and passages, add notes, group by topic; this is the investigation workspace	notes table plus drag-and-drop cards	P2
11	Contradiction matrix	Grid of documents against documents; red cells where they disagree, click to see why	Aggregate conflicts table	P2
12	Auto-summary on upload	After upload the app lists key facts, parties, dates, and detected anomalies without any question	One batch LLM call over extracted facts	P2
13	Suggested follow-up questions	Three clickable next questions, for example 'Do any files disagree on the delivery date?'	LLM call using answer, conflicts, and missing info	P2
14	Duplicate and version detection	'This looks like a newer version of Contract.pdf' with a text diff view	sha256 plus simhash plus difflib	P3
15	Hindi and English support	Ask in Hindi or English; OCR reads both scripts	Multilingual embeddings, Tesseract hin model	P3
16	Audit trail and reproducibility	'Show how this answer was produced': chunks, prompt version, model	audit_log table and a drawer in the UI	P3
17	Built-in evaluation dashboard	Page showing accuracy, citation precision, conflict recall, abstention accuracy on the golden set	eval/run_eval.py exposed via /eval	P3
18	PII redaction toggle	Mask phone, email, and ID numbers in answers and exports	Regex plus spaCy	P3

Recommended minimum for a winning submission: features 1 to 7 (all P1) plus 10 (Evidence Board) and 17 (evaluation dashboard). Feature 17 is cheap and strongly supports the 'Testing, Edge Cases and Reliability' score because judges can see measured results.

9. API Specification

Method and path	Purpose	Request / response summary
POST /documents	Upload one or many files	multipart files; returns list of {id, filename, status, doc_type, reliability}
GET /documents	List documents	returns documents with trust score and processing status
GET /documents/{id}/file	Serve original file	binary stream for the viewer
GET /documents/{id}/chunks?page=n	Chunks and boxes for a page	used for highlighting
DELETE /documents/{id}	Remove a document and its vectors	204 on success
POST /ask	Ask a question	{question, doc_ids?, conversation_id?} returns {answer, citations[], conflicts[], confidence{score, band, signals}, verified_claims, missing_info[], follow_ups[]}; supports SSE streaming
GET /conflicts	All conflicts across the case	returns contradiction matrix data
GET /entities?q=	Entity list and search	returns entities with document references
GET /timeline	Dated events	returns events sorted by date with doc and conflict flags
POST /notes, GET /notes	Evidence Board items	create, pin, and list notes tied to answers
POST /report	Generate report	{format: pdf md} returns file
GET /audit/{message_id}	Explain an answer	returns chunks, prompts, model, scores
POST /eval/run, GET /eval/latest	Run and read evaluation	returns metric summary
GET /health	Health check	{status: ok}

10. Frontend and UX Specification

10.1 Screens

- **Upload screen:** drag-and-drop zone, per-file progress and status chips (Extracting, OCR, Indexing, Ready, Failed), and a 'Load demo case' button that loads the prepared dataset in one click.
- **Workspace (main screen), three columns:** left = document list with trust cards and entity search; centre = chat with answers, confidence badge, Conflict Panel, claims verified count, follow-up chips; right = source viewer with page jump and highlight.
- **Tabs on the workspace:** Chat, Timeline, Entities, Contradiction Matrix, Evidence Board, Evaluation.
- **Report screen:** preview and download buttons for PDF and Markdown.

10.2 UX rules

- Show the answer streaming, then citations and badges as soon as available. Never show a blank spinner for more than two seconds without a status line such as 'Searching 6 documents'.
- Conflicts use a distinct red-bordered card above the answer text. Uncertainty uses amber. Verified content uses green. Do not rely on colour alone; add icons and text labels.
- Every empty state tells the user what to do next. Every error is human-readable and never exposes stack traces.

- Layout must be usable on a laptop projector at 1280 x 720 and acceptable on mobile width.

11. Demo Dataset Specification

The demo dataset is a fictional vendor dispute: Orion Interiors (buyer) ordered office furniture from Northwind Supplies Pvt Ltd (seller). Create these six documents. Conflicts and gaps are planted on purpose so every feature can be demonstrated.

File	Format	Key content (planted facts)
1_Supply_Contract.pdf	Native PDF, 4 pages	Signed 10 Jan 2026. Contract value INR 5,00,000 plus 18% GST. 50% advance (INR 2,50,000). Delivery by 15 Feb 2026. Late penalty 1% of order value per week. No warranty clause (the planted gap).
2_Invoice.jpg	Scanned photo (slightly skewed)	Invoice INR 5,90,000 total including GST. States advance received: INR 2,00,000 (conflicts with contract 2,50,000). Tests OCR.
3_Email_Vendor.docx	DOCX email	Vendor says goods dispatched 12 Feb and delivered on 20 Feb 2026.
4_Email_Client.docx	DOCX email	Buyer says goods were actually received on 28 Feb 2026 and are damaged. Asks for penalty.
5_Meeting_Notes.txt	Plain text	Vague line: 'payment will be settled soon.' No amount or date (tests uncertainty).
6_Payment_Receipt.png	Low-quality phone photo	Shows a payment of INR 2,50,000 on 11 Jan 2026. Intentionally blurry so the reliability score drops.

11.1 Scripted demo questions and expected behaviour

Question	Type	Expected result
What is the total contract value?	Easy factual	INR 5,00,000 plus 18% GST, cited to contract p.1; invoice total 5,90,000 is consistent. High confidence.
When was the furniture delivered?	Conflict	Contract says by 15 Feb; vendor email says 20 Feb; client email says 28 Feb. Conflict Panel lists the three values. Medium or low confidence with a banner.
How much advance was paid?	Numeric conflict	Invoice says 2,00,000; contract terms and receipt say 2,50,000. Cross-check flags the mismatch; receipt trust score is lower due to blur.
What is the warranty period?	Unanswerable	Abstains: not found; missing information: warranty terms. Suggests asking the vendor or checking for an annexure.
How many days was delivery late and what penalty applies?	Multi-document	Combines contract penalty (1% per week) with delivery dates; shows the result for each date version (5 days or 13 days late) and notes the dispute. Never gives one confident number.
When will the remaining payment be made?	Vague source	Reports that only 'soon' is stated in meeting notes, no date; low confidence.

12. Testing and Evaluation

12.1 Golden evaluation set

Create eval/golden_set.json with at least 25 labelled questions over the demo case: expected answer facts, expected source documents, whether a conflict should be flagged, and whether the system should abstain. run_eval.py reports:

- **Answer accuracy** (matches expected facts), **citation precision** (cited passages actually support the claim), **conflict recall** (planted conflicts found), **conflict precision** (no false alarms on consistent facts), **abstention**

accuracy (refuses unanswerable questions, does not refuse answerable ones).

12.2 Edge-case suite (must be automated)

Case	Expected behaviour
Empty upload / zero-byte file	Clear error, no crash
Unsupported file type (.exe, .zip)	Rejected with a friendly message
Password-protected or corrupted PDF	Status failed with reason; other files still process
Scanned page with unreadable text	OCR quality flagged; confidence lowered; user warned
Exact duplicate upload	Detected by sha256; skipped with notice
Near-duplicate / newer version	Flagged as possible new version (P3 feature)
Question with no relevant content	Abstains with missing-info list
Question that assumes a false fact ("Why did the contract include a warranty?")	Corrects the premise: no warranty clause found
Very long document (100+ pages)	Processes without timeout; progress shown
Prompt injection inside a document ('ignore previous instructions')	Treated as plain text; never obeyed
LLM returns invalid JSON	Retry once, then fall back to safe plain answer with a Low badge
LLM or OCR service unavailable	Graceful error and retry option
Mixed Hindi and English text	Indexed and answerable (P3 feature)

12.3 Test types

- Unit tests for chunker, money/date normaliser, fact cross-checker, confidence formula, and RRF merging.
- Integration tests for the full upload then ask flow using the demo data; assert that each scripted question in 11.1 yields the expected conflict or abstention.
- One Playwright smoke test: upload demo case, ask the delivery question, click a citation, confirm the highlight appears.
- Save results as docs/test_evidence.md; this goes directly into the submission.

13. Configuration, Security, and Privacy

```
# .env.example
LLM_PROVIDER=anthropic          # anthropic | openai | gemini | ollama
LLM_API_KEY=your_key_here
LLM_MODEL=your_model_name
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
RERANK_MODEL=BAAI/bge-reranker-base
OCR_LANGS=eng+hin
MAX_FILE_MB=25
ABSTAIN_THRESHOLD=0.25
TOP_K_RETRIEVE=20
TOP_K_FINAL=8
DATABASE_URL=sqlite:///./data/app.db
```

- Never commit API keys. Validate file types and sizes server-side. Sanitise filenames. Serve uploaded files only through the API, never from a public folder.
- Treat all document text as untrusted data: it is inserted into prompts inside clearly delimited blocks and the system prompt states that instructions inside passages must be ignored.
- Make LLM calls through one client module so the provider can be switched with a single environment variable.

- Add a visible banner if the app is running with sample data so judges are never confused about what is real.

14. Build Order and Definition of Done

Build in this order. Each step must work and be demonstrable before the next begins. No time estimates are given; move through the steps as fast as quality allows.

Step	Build	Done when
1	Project skeleton, config, database, LLM client, health check	App starts; /health returns ok; LLM test call succeeds
2	Ingestion: PDF, DOCX, TXT, image OCR, chunker, indexing	All six demo files reach status Ready with page-accurate chunks
3	Hybrid retrieval, reranker, grounded answer with citations	Easy factual question answered with correct citations
4	Source viewer with page jump and passage highlight	Clicking a citation highlights the exact passage
5	Facts extraction and deterministic cross-check	Advance-amount and delivery-date mismatches detected without the LLM
6	LLM conflict detection and Conflict Panel UI	Delivery-date question shows three-way conflict
7	Confidence scoring, badge, abstention, missing-info list	Warranty question abstains; delivery question shows reduced confidence
8	Claim-level verification	Verified-claims counter works; an injected unsupported claim is caught
9	Entities, timeline, trust cards, auto-summary, follow-up questions	All visible in the workspace and driven by real data
10	Evidence Board, contradiction matrix, report export	Report PDF contains questions, answers, citations, conflicts
11	Evaluation harness, edge-case suite, evaluation dashboard	Golden set results displayed; edge cases pass
12	Optional P3 features, polish, deployment, documentation	Public demo URL works; README complete

14.1 Acceptance checklist

- All six demo documents upload and become searchable.
- All six scripted questions in Section 11.1 produce the expected behaviour.
- Every answer shows citations, a confidence badge, and (where relevant) a Conflict Panel.
- The warranty question abstains instead of guessing.
- Report export works and opens correctly.
- Edge-case suite passes and results are saved in docs/test_evidence.md.
- A new user can run the project using only the README and docker compose up.

15. Submission Deliverables

Deliverable	Contents
Working project and live demo	Deployed URL where practical, plus local run instructions
Source repository with README	Setup steps, environment variables, how to load demo data, how to run tests
Architecture diagram and explanation	docs/architecture.png and docs/decisions.md describing why each technology was chosen
Core workflow demonstration	Upload, ask, citation highlight, conflict, abstention, report export; rehearsed three-minute script

Deliverable	Contents
Testing evidence	docs/test_evidence.md with edge-case results and evaluation metrics
Known limitations and future work	For example: OCR errors on handwriting, no cross-language conflict detection, no user accounts
Disclosure of external APIs, datasets, and AI-assisted components	Name the LLM provider, embedding and reranker models, OCR engine, and state which code was AI-assisted

16. Master Hand-off Prompt (copy and paste)

Give this prompt together with this document to the AI coding agent or agency:

You are a senior full-stack and ML engineer. Read the attached "Document Investigator - Complete Build Specification" fully before writing code. Build the complete product exactly as specified: FastAPI backend, Next.js frontend, SQLite plus ChromaDB, hybrid retrieval with reranking, grounded answers with clickable page-level citations, LLM and deterministic conflict detection with a Conflict Panel, confidence scoring with abstention, and claim-level verification.

Follow the build order in Section 14 and do not skip steps. After each step, run the tests and confirm the "Done when" condition. Create the demo dataset from Section 11 (all six files, with the planted conflicts) and make the six scripted questions work. Implement the P1 and P2 standout features from Section 8, then P3 if stable.

Constraints: every answer must cite sources, never hide a conflict, and abstain when evidence is weak. Keep all prompts in one file, keep the LLM provider switchable by an environment variable, and never commit secrets. Produce the README, architecture diagram, test evidence, known limitations, and AI-usage disclosure listed in Section 15. If something is ambiguous, use the default stated in this document and note the assumption in docs/decisions.md instead of asking.